isec
Politécnico de Coimbra

Conhecimento e Raciocínio

Ano Letivo 2025 – 2026

Trabalho Prático

CBR & Redes Neuronais Feedforward

Docente das aulas práticas: Anabela Borges Simões

Trabalho realizado por:

2023155012 – André Pessoa Tavares – P5

2024140892– Rodrigo Salgueiro da Silva Ideia - P6

Indices

- 1 - Introdução 1
- 2 - Tratamento do dataset 1
- 3 - Raciocínio Baseado em casos 3
- 4 - Redes Neuronais 8

5. Comparar resultados do CBR e RN 16

6 - Conclusões 17

1 - Introdução

O presente trabalho prático, desenvolvido no âmbito da unidade curricular de Conhecimento e Raciocínio do curso de Licenciatura em Engenharia Informática, tem como objetivo central a conceção, implementação e análise de sistemas inteligentes aplicados ao diagnóstico de falhas em motores elétricos assíncronos.

O projeto explora uma abordagem híbrida que combina o paradigma de Raciocínio Baseado em Casos (CBR) com modelos de Redes Neuronais Artificiais (ANN). Através da implementação do ciclo de Aamodt e Plaza — que compreende as fases de Recuperação (*Retrieve*), Reutilização (*Reuse*), Revisão (*Revise*) e Retenção (*Retain*) — o sistema visa emular o raciocínio pericial humano, utilizando a experiência passada para solucionar novos problemas de manutenção preditiva.

Complementarmente, é abordada a vertente de *Deep Learning*, focada no desenvolvimento de redes *Feedforward* para a classificação de avarias. Esta componente permitiu uma investigação aprofundada sobre o impacto de diferentes hiperparâmetros — tais como topologias, funções de ativação e algoritmos de treino — na capacidade de generalização do modelo.

Previamente à fase de raciocínio e modelação, o trabalho dedica especial atenção ao Pré-processamento de Dados, assegurando o tratamento rigoroso de *missing values* e a normalização técnica dos atributos, garantindo a integridade e a coerência matemática necessária para a obtenção de resultados de elevada precisão num contexto industrial.

2 - Tratamento do dataset

2.1 Conversão de Variáveis Categóricas

Relativamente à transformação dos dados categóricos para o modelo, adotámos as seguintes lógicas de conversão para preservar as relações inerentes aos valores originais:

- **Maintenance Level:** Tratando-se de um atributo ordinal, decidimos mapear os seus três estados para um intervalo contínuo entre 0 e 1. Atribuímos o valor **0** ao nível "Low" (equipamento sem ou com pouca manutenção), **0.5** ao nível "Medium" (manutenção intermédia) e **1** ao nível "High" (manutenção regular e adequada).
- **Operating Mode:** Para refletir o nível de carga e esforço do motor, definimos uma escala crescente. O valor **0** representa o motor parado ("Idle"), o valor **0.5**

indica o funcionamento natural do equipamento ("Normal") e o valor **1** assinala uma situação de sobrecarga mecânica ("Overload").
- **Cooling Type:** Como se trata de uma variável de dois estados, optámos por uma codificação binária. Atribuímos o valor **1** aos motores arrefecidos a óleo ("Oil") e **0** aos sistemas arrefecidos a ar ("Air").
- **Sensor Status:** Seguindo a mesma lógica binária, definimos o valor **1** para indicar um funcionamento sem anomalias ("OK") e **0** para assinalar a ocorrência de um problema ou aviso ("Warning").

2.2 Tratamento de Valores Omissos nos Atributos de Entrada

Para o tratamento de valores em falta (ausência de dados) nos atributos de entrada, adotámos diferentes estratégias estatísticas com base na distribuição e na natureza específica de cada variável:

- **Média aritmética:** Aplicada a atributos numéricos que apresentam uma distribuição tendencialmente equilibrada e ausência de *outliers* (valores extremos) significativos que pudessem enviesar a métrica. Esta abordagem foi utilizada nas colunas `efficiency` e `load_val`.
- **Mediana:** Escolhida para atributos numéricos caracterizados por uma elevada variabilidade ou suscetibilidade a valores extremos. Sendo a mediana uma medida de tendência central estatisticamente robusta face a *outliers*, garante uma imputação mais fiável. Foi aplicada às colunas `temperature`, `vibration`, `rotation_speed`, `voltage`, `current`, `pressure`, `torque` e `noise_level`.
- **Moda:** Selecionada para a imputação de dados nas variáveis de natureza categórica (previamente convertidas para formato numérico), garantindo que os valores introduzidos correspondem à classe mais frequente no conjunto de dados. Aplicou-se às colunas `maintenance_level`, `operating_mode`, `cooling_type` e `sensor_status`.

2.3 Normalização dos Dados

De forma a assegurar que atributos com grandezas físicas e escalas muito distintas (por exemplo, a velocidade de rotação na ordem dos milhares e a vibração na ordem das décimas) não dominassem enviesadamente os cálculos matemáticos do sistema, procedeu-se à normalização global das 14 variáveis de entrada. Utilizou-se a abordagem baseada na divisão de cada observação pelo valor máximo absoluto encontrado na respetiva coluna (V/Vmax), garantindo que todos os valores ficassem contidos no intervalo numérico entre **0** e **1**.

2.4 Codificação e Imputação da Variável Alvo (Diagnóstico)

O último passo de pré-processamento incidiu sobre o diagnóstico final dos motores (variável `class_cat`). Numa primeira fase, as classificações em formato de texto foram convertidas num mapeamento numérico discreto: "Normal" (**1**), "Electrical Failure" (**2**) e "Mechanical Failure" (**3**).

Posteriormente, verificou-se a existência de casos na base de dados original em que o diagnóstico final estava omisso (valores nulos). Para a imputação rigorosa destes

casos, em vez de se recorrer a métricas estatísticas simples, optou-se pela implementação de uma heurística avançada fundamentada nos princípios do Raciocínio Baseado em Casos (CBR). Para cada motor sem diagnóstico conhecido, o algoritmo calculou a distância linear ponderada em relação a todos os casos rotulados. Através da identificação da distância mínima global (lógica do vizinho mais próximo ou *1-NN*), o sistema imputou automaticamente a classe de diagnóstico correspondente ao caso histórico mais semelhante, garantindo assim a total integridade e preenchimento da *Case Library* antes do início da avaliação.

● 3 - Raciocínio Baseado em casos

3.1 Implementação

A implementação do sistema de Raciocínio Baseado em Casos (Tarefa 3.2) teve como objetivo diagnosticar falhas em motores elétricos tirando partido da experiência acumulada em diagnósticos passados. A arquitetura do sistema foi desenvolvida em MATLAB e rigorosamente estruturada em torno do ciclo clássico dos 4 R's delineado por Aamodt e Plaza: *Retrieve* (Recuperar), *Reuse* (Reutilizar), *Revise* (Rever) e *Retain* (Reter).

Para avaliar o sistema num cenário prático (através do *script* principal `cbr.m`), instanciou-se manualmente um "novo caso" de teste, simulando um motor com desgaste severo nos rolamentos (Falha Mecânica - Classe 3).

Justificação do Cenário de Teste (Instanciação Manual): Para garantir a validade desta simulação prática, o vetor de dados instanciado manualmente (`novo_caso_array`) foi desenhado para espelhar de forma rigorosa a assinatura física deste tipo de avaria mecânica. Os valores brutos atribuídos justificam-se pelas seguintes premissas de engenharia:

- **Vibração e Ruído Elevados:** O nível de vibração (1.29) e o nível de ruído acústico (73.69) foram intencionalmente colocados em patamares altos, constituindo o sintoma primário e mais direto da degradação das esferas ou pistas do rolamento.
- **Sobreaquecimento por Fricção:** A temperatura registada (78.21°C) encontra-se num limiar de alarme, refletindo o calor excessivo gerado pelo atrito mecânico interno.
- **Comportamento Eletromecânico:** Embora a tensão de alimentação esteja estável (222.44V), a corrente absorvida (18.55A) está elevada, e a velocidade de rotação apresenta escorregamento (1442.02 RPM). Isto indica que o motor está a extrair mais energia da rede numa tentativa de vencer a resistência mecânica oferecida pelos rolamentos danificados, operando num estado de sobrecarga e com uma eficiência degradada (0.78).

Os dados empíricos em bruto foram instanciados no código (**Figura 1**) e, em seguida, automaticamente normalizados em tempo real para a escala da *Case Library*.

```
% NOVO CASO MANUAL (Valores Brutos)
novo_caso_array =   [78.21, 1.29, 1442.02, 222.44, 18.55, 5.22, 73.69, 0.78, 55.6, 106.99, 0.5, 1, 0, 1];

nomes_colunas = {'temperature', 'vibration', 'rotation_speed', 'voltage', ...
             'current', 'pressure', 'noise_level', 'efficiency', ...
             'load_val', 'torque', 'maintenance_level', 'operating_mode', ...
             'cooling_type', 'sensor_status'};

novo_caso = array2table(novo_caso_array, 'VariableNames', nomes_colunas);

% Target: Falha Mecânica (Classe 3)
novo_caso.class_cat = 3;
```

Figura 1: Instanciação manual do novo caso de teste e definição da classe alvo no script cbr.m.

O fluxo de execução demonstrou a correta articulação das 4 fases do Raciocínio Baseado em Casos:

1. Fase de Recuperação (*Retrieve*) A função `retrieve.m` foi concebida para procurar no *dataset* de treino os motores históricos mais semelhantes ao novo caso apresentado. Para a métrica de similaridade, optou-se pelo cálculo da Distância Linear Ponderada, complementada por um limiar de aceitação (*similarity threshold*) restritivo de 90%.

Justificação Técnica do Vetor de Pesos (Conhecimento Perito): A atribuição heurística dos pesos (`[3, 3, 2, 2, 3, 2, 1, 1, 1, 2, 1, 1, 1, 1]`) fundamentou-se na relevância física e eletromecânica de cada variável para a deteção de avarias:

- **Pesos Críticos (Peso 3):** Atribuídos à Temperatura (indicador de sobreaquecimento), Vibração (desgaste mecânico) e Corrente (curto-circuitos ou isolamento degradado).
- **Pesos Elevados (Peso 2):** Atribuídos à Velocidade de Rotação, Tensão, Pressão e Torque. São métricas que revelam esforço mecânico real e flutuações de alimentação.
- **Pesos Base (Peso 1):** Atribuídos a variáveis ruidosas, de contexto ou categóricas, como o Nível de Ruído, Eficiência, Carga, Estado do Sensor e Modo de Operação. Auxiliam no enquadramento, mas não possuem variância isolada suficiente para diagnosticar a causa raiz da falha em tempo real.

2. Fase de Reutilização (*Reuse*) Na fase de reutilização das soluções dos casos passados (`reuse.m`), adotou-se uma **abordagem híbrida e avançada**, integrando o CBR com Redes Neuronais Artificiais. Em vez de realizar uma cópia direta da temperatura do motor recuperado, o sistema treina dinamicamente uma Rede Neuronal *Feedforward* de regressão (com 10 neurónios). A rede aprende as relações não-lineares entre a Vibração, Rotação e Tensão para prever a Temperatura. No teste efetuado, a rede concluiu que a temperatura ideal estimada para o motor seria de **0.7348**, contrastando com os **0.8398** registados originalmente pelo sensor.

3. Fase de Revisão (*Revise*) O processo de revisão (`revise.m`) foi desenhado para incorporar o princípio de *Human-in-the-loop*. O sistema apresentou a nova temperatura gerada matematicamente e interpelou o operador humano via terminal. Na simulação realizada, optou-se por rejeitar a sugestão, mantendo a temperatura empírica original do caso.

4. Fase de Retenção (*Retain*) Após o diagnóstico, a fase final (`retain.m`) questionou o utilizador sobre o desejo de incorporar este motor na Base de Casos. Para efeitos desta demonstração, optou-se por não reter o caso, preservando a integridade da *Case Library* original.

```
Based on the attributes Vibration, Rotation Speed and Voltage of the case base,
the estimated temperature for the new case is 0.7348, instead of 0.8398.

Reuse phase completed...

Starting revise phase...

Update your engine temperature with the new estimated value? (y/n)
Option: n
Original temperature maintained.

Revise phase completed...

Starting retain phase...

Add the new case to the library? (y/n)
Option: n
New case was not added.

Retain phase completed...
```

Figura 2: Execução das fases de Reuse, Revise e Retain com interação do utilizador no terminal.

Esta demonstração provou que o sistema não atua como uma "caixa negra", mas sim como um assistente interativo, que sugere adaptações suportadas por Inteligência Artificial, deixando a decisão final e curadoria dos dados ao encargo do especialista.

3.2 Resultados

Para além da validação prática do ciclo num caso individual, desenvolveu-se um módulo de testes independente (`cbr_teste.m`) focado na validação global estatística da fase de Recuperação (*Retrieve*). O sistema foi submetido a uma bateria de testes com 30 motores do conjunto de teste (simulando a identificação do vizinho mais próximo *1-NN*).

Metodologia da Bateria de Testes: O racional por detrás da escolha destas 6 configurações de pesos visou testar os limites do algoritmo face a diferentes cenários de disponibilidade de dados e qualidade de *Feature Selection*:

1. **Pesos Iguais (Cofiguração por Defeito):** Atua como o ponto de controlo. Demonstra o desempenho base quando não é introduzido qualquer conhecimento prévio sobre o domínio, assumindo que os 14 atributos contribuem com exata igualdade.
2. **Conhecimento Perito:** Avalia a introdução de *Domain Knowledge*. Pretende validar se a afinação empírica da métrica refina e estabiliza as previsões face à Configuração por Defeito.

3. **Foco Extremo na Física do Motor:** Testa a suficiência das variáveis numéricas contínuas. Ao anular atributos categóricos, verifica se a física bruta (pressões, temperaturas) contém informação isolada suficiente para prever a falha.
4. **Foco Exclusivo nos Alarmes:** Explora a redundância do *dataset*. Força o CBR a classificar casos olhando apenas para variáveis categóricas de painel. É expectável uma queda na precisão gerada por empates matemáticos (*ties*).
5. **Teste de Stress (Sem Sensores Chave):** Simula um cenário de falha parcial de instrumentação. Foram anuladas variáveis cruciais (Velocidade e Tensão) para avaliar a robustez à falta de informação descritiva.
6. **Degradação (Apenas Ruído):** Testa o limite inferior de precisão. Ao forçar o sistema a diagnosticar falhas baseando-se exclusivamente no Ruído Acústico, demonstra-se as consequências catastróficas de uma má etapa de seleção de atributos.

Os resultados obtidos encontram-se sumarizados na Tabela 1.

	Array de Pesos	Resultados
Teste1 - Pesos Iguais (Configuração por Defeito)	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]	100,00% (30/30)
Teste2 - Pesos do Guiao (Conhecimento Perito)	[3, 3, 2, 2, 3, 2, 1, 1, 1, 2, 1, 1, 1, 1]	100,00% (30/30)
Teste3 - Foco Extremo na Fisica do Motor	[5, 5, 1, 1, 5, 5, 1, 1, 1, 5, 0, 0, 0, 0]	100,00% (30/30)
Teste4 - Foco Exclusivo nos Alarmes	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 10, 10, 10, 10]	40.00% (12/30)
Teste5 - Teste de Stress (Sem Sensores Chave)	[1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 5, 5, 5, 5]	100% (30/30)
Teste6 - Degradação (Apenas Ruído)	[0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]	86.67% (26/30)

Tabela 1: Avaliação de Precisão (Accuracy) do sistema CBR de acordo com as 6 configurações de pesos testadas.

3.3 Análise Detalhada de Resultados

- #### *3.3.1 A Lógica da Adaptação na Fase de Reuse*

  Um dos momentos mais demonstrativos da inteligência do sistema ocorre na fase de Reuse. Ao analisarmos o novo caso inserido (um motor com falha mecânica), o sensor registava uma temperatura normalizada de **0.8398**. No entanto, a Rede Neuronal estimou que o valor ideal deveria ser **0.7348**.
  Esta discrepância de aproximadamente 10% é fundamental para o diagnóstico:
  - **Filtro de Anomalias:** A temperatura real (0.8398) estava inflacionada devido ao atrito excessivo dos rolamentos degradados. A Rede Neuronal, tendo sido treinada com a base histórica, "sabe" qual seria a temperatura esperada para um motor saudável a operar com aquela vibração e carga.

    - **O Valor da Adaptação:** Se o sistema se limitasse a copiar o valor do caso mais parecido, estaria a ignorar as especificidades elétricas (voltagem e rotação) do motor atual. Ao sugerir 0.7348, o sistema está a dizer ao perito: "Baseado na física dos outros motores, este motor está a aquecer 0.10 acima do que a sua mecânica atual justificaria". Isto confirma que a temperatura alta é um **sintoma de falha** e não apenas uma consequência natural da operação.

- *3.3.2 Análise Comparativa das Configurações de Pesos*

A variabilidade dos resultados na Tabela 1 permite extrair conclusões profundas sobre a importância de cada atributo:

- **Configurações de Sucesso (1, 2, 3 e 5 - 100% de Precisão):**
    - O facto de estas configurações atingirem a precisão máxima indica que o dataset possui classes (Normal, Elétrica, Mecânica) com assinaturas físicas muito distintas.
    - A **Configuração 3 (Física Extrema)** prova que sensores de vibração e temperatura são "autossuficientes": mesmo ignorando todos os alarmes e dados de manutenção, a matemática dos sensores é suficiente para separar as falhas.
    - A **Configuração 5 (Teste de Stress)** é a mais surpreendente, pois mesmo retirando sensores chave como Rotação e Voltagem, o sistema manteve os 100%. Isto demonstra a **redundância positiva** do sistema: a falha elétrica que a voltagem detetaria acaba por se manifestar também na temperatura e na corrente, permitindo ao CBR recuperar o diagnóstico correto por vias alternativas.
- **O Fracasso da Configuração 4 (Foco em Alarmes - 40% de Precisão):**
    - Esta queda drástica para menos de metade da precisão explica-se pela **baixa variância dos dados categóricos**. Variáveis como "Sensor Status" ou "Operating Mode" têm poucos estados (0, 0.5, 1).
    - Num dataset com centenas de motores, muitos terão exatamente o mesmo perfil de alarmes, mas estados de saúde diferentes. Ao usar apenas estes pesos, o cálculo da distância gera inúmeros "empates matemáticos", levando o sistema a recuperar casos que parecem iguais no painel, mas que são fisicamente opostos, resultando em diagnósticos errados.
- **A Fragilidade da Configuração 6 (Apenas Ruído - 86.67% de Precisão):**
    - Embora 86% pareça um valor alto, numa linha de produção significa que 4 em cada 30 motores seriam mal diagnosticados.
    - O nível de ruído é um preditor fraco porque é "sujo": pode ser influenciado pelo ambiente da fábrica ou por vibrações externas. Esta descida na precisão prova que o CBR não pode confiar em variáveis isoladas que não tenham uma correlação direta e pura com a integridade do motor.

3.4 Conclusões Finais

A análise conjunta da demonstração prática e da validação quantitativa evidencia que a eficácia de um sistema CBR em cenários de manutenção industrial está estritamente dependente da afinação heurística da sua métrica de similaridade.

O projeto demonstrou que o **Conhecimento de Domínio** (atribuição de pesos diferenciados) é superior ao processamento cego de dados. A capacidade do sistema em "desempatar" casos complexos depende de dar o peso certo aos "Reis do Diagnóstico": a Temperatura e a Vibração.

Finalmente, a arquitetura híbrida com Redes Neuronais provou ser o elemento diferenciador. Enquanto o CBR clássico é um sistema de memória, a inclusão da ANN transforma-o num sistema de **análise preditiva**, capaz de entender que a solução para um problema novo não é apenas uma cópia do passado, mas sim uma adaptação inteligente baseada em padrões universais de engenharia.

● 4 - Redes Neuronais

4.1 Implementação

A implementação das redes neuronais feedforward seguiu uma lógica modular para permitir a automação de testes exigida pelo enunciado.

- **Preparação de Dados:** A matriz de entrada foi construída a partir dos 14 atributos dos sensores, aplicando-se a transposta para alinhar os atributos às linhas e os exemplos às colunas, conforme exigido pela arquitetura clássica do MATLAB.

- **Codificação do Alvo (One-Hot Encoding):** Para a classificação das três falhas (Normal, Elétrica e Mecânica), implementou-se o *One-Hot Encoding*. Esta escolha foi crítica para evitar o "absurdo matemático" de tratar classes categóricas como uma hierarquia numérica, garantindo que a rede possui três neurónios de saída independentes.

- **Função de Treino e Avaliação:** Criou-se a função '*treinarRede*' que encapsula a lógica de configuração da *feedforwardnet*, definição de funções de ativação e rácios de divisão de dados.

- **Rigor Estatístico:** De modo a mitigar o impacto da inicialização aleatória dos pesos, cada configuração foi submetida a **10 repetições**, calculando-se a média das precisões global e de teste para uma análise mais robusta. A avaliação de acerto baseou-se na função max(), que identifica o neurónio de saída com maior convicção e compara-o com o alvo real.

4.2 Resultados

A análise dos resultados permitiu identificar padrões claros no comportamento da rede face à variação dos hiperparâmetros.

- **Tipologia de defeito:** A rede neuronal base foi configurada com 1 camada oculta de 10 neurónios, algoritmo trainlm, ativações tansig-purelin e divisão de dados a 70/15/15 (dividerand). Esta configuração de referência estabeleceu um patamar de desempenho muito elevado, com 99,86% de precisão global e 99,56% de precisão de teste.

	Número de camadas escondidas	Número de neurónios	Funções de ativação	Função de treino	Divisão dos exemplos		Precisão Global	Precisão Teste
Configuração por defeito	1	10	tansig, purelin	trainlm	dividerand = {0.7, 0.15, 0.15}		99,86	99,56

Tabela 2 – Configuração de Defeito

- **Diferentes Topologias:** Observou-se que o aumento de neurónios (ex: configuração 25) ou de camadas (ex: [10 10]) nem sempre resulta numa melhor generalização. Redes excessivamente complexas tenderam a apresentar precisões de treino próximas dos 100% mas com uma quebra na precisão de teste, um indicador clássico de ***overfitting*** (a rede decorou os dados em vez de aprender os padrões).

O número e dimensão das camadas encondidas								
Conf1	2	5, 5	tansig, tansig, purelin	trainlm	divideran d = {0.7, 0.15, 0.15}		99.88	99.68
Conf2	2	10,10	tansig, tansig, purelin	trainlm	divideran d = {0.7, 0.15, 0.15}		99.87	99.64
Conf3	1	25	tansig, purelin	trainlm	divideran d = {0.7, 0.15, 0.15}		99.86	99.61
Conf4	2	8,4	tansig- tansig- tansig- purelin	trainlm	divideran d = {0.7, 0.15, 0.15}		99.82	99.40

Tabela 3 – Número de Dimensões e Camadas

- **Funções de Ativação:** A alteração das funções de ativação nas camadas oculta e de saída não revelou um impacto tão significativo no desempenho da rede como a modificação de outros hiperparâmetros, mantendo a maioria dos testes acima dos 99% de precisão. A arquitetura demonstrou a sua melhor capacidade de generalização ao utilizar a combinação *logsig* na camada oculta e *purelin* na camada de saída, culminando num resultado de teste de 99,67%. Em contrapartida, o pior registo ocorreu com a configuração *logsig, logsig*, cuja precisão de teste decaiu para 97,88%. Esta ligeira degradação sugere que a imposição de limites não-lineares severos em ambas as camadas pode dificultar a convergência fina do algoritmo de treino perante estes dados específicos.

As funções de ativação influenciam o desempenho?								
Conf1	1	10	logsig, purelin	trainlm	divideran d = {0.7, 0.15, 0.15}		99.88	99.67
Conf2	1	10	tansig, logsig	trainlm	divideran d = {0.7, 0.15, 0.15}		99.86	99.60
Conf3	1	10	tansig,ta nsig	trainlm	divideran d = {0.7, 0.15, 0.15}		99.85	99.55
Conf4	1	10	tansig,sof tmax	trainlm	divideran d = {0.7, 0.15, 0.15}		99.82	99.52
Conf5	1	10	logsig,log sig	trainlm	divideran d = {0.7, 0.15, 0.15}		97,96	97,88
Conf6	1	10	logsig,sof tmax	trainlm	divideran d = {0.7, 0.15, 0.15}		99.77	99.55

Tabela 4 – Funções de ativação

- **Funções de Treino:** A escolha do algoritmo de treino revelou-se o fator mais determinante no desempenho da rede neuronal. A função *Gradient Descent* clássica (traingd) apresentou uma performance consideravelmente inferior às restantes, alcançando apenas 85,83% de precisão de teste, o que indica dificuldades em convergir eficazmente perante a complexidade destes dados. Em forte contraste, as funções que incorporam técnicas de otimização mais avançadas apresentaram resultados de excelência. O trainbr (Regularização Bayesiana) atingiu 99,43% de teste, enquanto o trainscg e o traingda registaram 99,40% e 99,17%, respetivamente. Conclui-se, contudo, que o algoritmo de *Levenberg-Marquardt* (trainlm), utilizado na arquitetura base, continua a ser o mais robusto e adequado para este problema de classificação.

A função de treino influencia o desempenho?								
Conf1	1	10	tansig, purelin	traingd	divideran d = {0.7, 0.15, 0.15}		86,39	85,83
Conf2	1	10	tansig, purelin	trainscg	divideran d = {0.7, 0.15, 0.15}		99.67	99.40
Conf3	1	10	tansig, purelin	trainbr	divideran d = {0.7, 0.15, 0.15}		99.91	99.43
Conf4	1	10	tansig, purelin	traingda	divideran d = {0.7, 0.15, 0.15}		99.23	99.17

Tabela 5 – Funções de Treino

- **Rácios de Divisão:** A experimentação com diferentes proporções de divisão do *dataset* revelou dinâmicas importantes sobre o fenómeno de *overfitting*. Contrariando a premissa de que "mais dados de treino equivalem a um modelo melhor", a configuração com 90% de dados alocados ao treino ([0.9, 0.05, 0.05]) obteve o pior desempenho de generalização da bateria de testes, com uma precisão de teste de apenas 99,32% (apesar da aparente precisão global de 99,91%). Ao invés, o modelo mais competente perante dados virgens surgiu ao aplicar a divisão [0.70, 0.10, 0.20], que garantiu o melhor resultado de teste (99,72%). Fica assim demonstrado que a preservação de um conjunto de teste generoso (20%) é fundamental para validar a verdadeira capacidade de aprendizagem da rede neuronal.

A divisão de exemplos pelos conjuntos influencia o								
Conf1	1	10	tansig, purelin	trainlm	divideran d = {0.8, 0.10, 0.10}		99.89	99.60
Conf2	1	10	tansig, purelin	trainlm	divideran d = {0.9, 0.05, 0.05}		99.91	99.32
Conf3	1	10	tansig, purelin	trainlm	divideran d = {0.50, 0.25, 0.25}		99.76	99.55
Conf4	1	10	tansig, purelin	trainlm	divideran d = {0.60, 0.20, 0.20}		99.81	99.57
Conf5	1	10	tansig, purelin	trainlm	divideran d = {0.70, 0.20, 0.10}		99.86	99.58
Conf6	1	10	tansig, purelin	trainlm	divideran d = {0.70, 0.10, 0.20}		99.87	99.72

Tabela 6 – Divisão dos conjutos

Normalização vs Não Normalização: A repetição das experiências com o *dataset* normalizado (onde as discrepâncias de grandeza matemática entre atributos foram eliminadas) permitiu retirar conclusões fundamentais sobre a estabilidade das diferentes arquiteturas. A análise cruzada entre as taxas de acerto globais e as respetivas matrizes de confusão revela três comportamentos distintos:

NÃO NORMALIZADO	Número de camadas escondidas	Número de neurónios	Funções de ativação	Função de treino	Divisão dos exemplos		Precisão Global	Precisão Teste		NORMALIZADO	
1ª Melhor	1	10	tansig, purelin	trainlm	dividerand = {0.70, 0.10, 0.20}		99.76	99.20		100	100
2ª Melhor	2	5,5	tansig, tansig, purelin	trainlm	dividerand = {0.7, 0.15, 0.15}		99.96	99.87		93.33	80
3ª Melhor	1	10	logsig, purelin	trainlm	dividerand = {0.7, 0.15, 0.15}		99.86	99.73		100	100
1ª Pior	1	10	tansig, purelin	traingd	dividerand = {0.7, 0.15, 0.15}		91.30	90.93		86.67	60
2ª Pior	1	10	logsig, logsig	trainlm	dividerand = {0.7, 0.15, 0.15}		67.02	65.87		96.67	80
3ª Pior	1	10	tansig, purelin	traingda	dividerand = {0.7, 0.15, 0.15}		99.52	99.07		100	100

Tabela 7 – Não Normalizado vs Normalizado

- **O Efeito de Otimização Perfeita:** Nas arquiteturas baseadas numa única camada oculta com algoritmos avançados (como o trainlm), a normalização dos dados serviu como o polimento final necessário. Como se pode observar na matriz de confusão abaixo, os erros residuais presentes no modelo não normalizado desapareceram por completo. A normalização homogeneizou o espaço de características, permitindo que a rede atingisse a perfeição preditiva (100% de acerto no conjunto de teste)



Figura 3 – Comparação das Matrizes Confusão

- **O Efeito de Resgate:** O impacto mais dramático e positivo da normalização ocorreu na configuração l*ogsig-logsig*. Analisando a matriz de confusão não normalizada (à esquerda), constata-se uma falha crítica na abstração do modelo: a rede foi incapaz de classificar corretamente a Classe 3 (0 acertos). Contudo, ao normalizar os dados, a rede foi "resgatada" desse colapso matemático. A matriz normalizada (à direita) demonstra uma recuperação notável, passando a identificar corretamente a grande maioria das amostras da Classe 3, o que elevou a sua precisão de teste de 65,87% para 80%



Figura 4 – O efeito de resgate da normalização na configuração logsig-logsig

- **O Efeito de Degradação:** Paradoxalmente, a normalização prejudicou arquiteturas específicas. A configuração com topologia mais complexa e a configuração treinada com *Gradient Descent* puro (traingd) sofreram quebras abruptas na precisão. A matriz de confusão da rede traingd normalizada evidencia esta degradação, mostrando uma clara confusão entre as Classes 2 e 3 face aos dados normalizados. Isto indicia que o esmagamento das escalas dos atributos removeu a variância natural de que estes algoritmos complexos necessitavam para separar corretamente as fronteiras de decisão, induzindo *overfitting*.



Figura 5 – Colapso de desempenho da função traingd perante dados normalizados.

Conclusão: A análise global comprova que a normalização é uma ferramenta indispensável para estabilizar o treino de redes com funções de ativação limitativas e para alcançar a perfeição em topologias padrão. Contudo, o seu uso não garante melhorias universais e pode, de facto, saturar precocemente redes excessivamente complexas ou modelos de descida de gradiente mais básicos.

4.3 Teste Final com o Dataset Teste

Para confirmar qual a rede mais fiável num cenário real, testámos as três melhores com um ficheiro de dados completamente novos e que a rede nunca tinha visto . A conclusão principal é clara: a solução mais simples é a mais eficaz.

- **Generalização Perfeita (1ª e 3ª Melhores Redes)** : As redes mais simples (com apenas 1 camada oculta de 10 neurónios) tiveram um desempenho impecável, atingindo **100% de acerto global**. Analisando as matrizes de confusão, não houve um único erro em nenhuma das três classes de falha.



Figura 6 – Matrizes Confusão da 1ª e 3ª Melhores Redes

4.4 A Falha da Rede Complexa

A rede mais complexa (com duas camadas ocultas) vacilou perante dados novos, e a sua precisão caiu para **93,33%**. Olhando para a matriz de confusão, percebemos exatamente onde o modelo hesitou:

- **Acertou sempre** nas Falhas Mecânicas (Classe 3).
- **Confundiu-se** entre a Classe 1 e a Classe 2: classificou erradamente um motor Normal como tendo Falha Elétrica, e vice-versa.

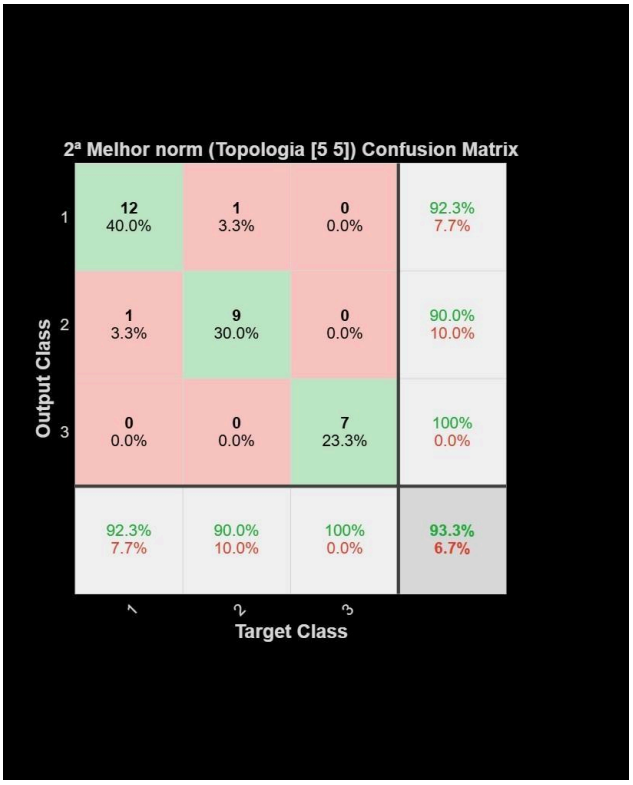


Figura 7 – Matriz Confusão da 2ª Melhor Rede

4.5 Conclusão da Validação:

Este teste final prova que adicionar mais camadas escondidas ([5 5]) prejudicou a rede. A rede complexa sofreu de *overfitting*: decorou demasiado os dados de treino e acabou por confundir falhas subtis na vida real. Fica provado que, para este diagnóstico industrial, redes de camada única são a opção mais segura e robusta.

5. Comparar resultados do CBR e RN

Olhando lado a lado para os dois sistemas de Inteligência Artificial que desenvolvemos, percebemos que não há um "vencedor absoluto", mas sim ferramentas diferentes que resolvem problemas de formas distintas no chão de fábrica:

- **Os Acertos (Eficácia Preditiva):** Ambos foram excelentes. Com a configuração certa, os dois sistemas chegaram aos 100% de precisão nos testes.

- **Como e quando falham:**

    - **O Ponto Fraco do CBR:** O CBR é como um assistente muito obediente, mas que precisa de ser bem guiado. Se não lhe dissermos exatamente quais são as variáveis mais importantes (*Feature Selection*), ele perde-se. Quando o forçamos a olhar apenas para os alarmes do painel (ignorando a física), a sua precisão caiu para 40%.
    - **O Ponto Fraco das Redes Neuronais:** As Redes são muito exigentes com a matemática. Se lhes dermos dados mal preparados (com grandezas não normalizadas) ou usarmos algoritmos de treino fracos, a rede não consegue aprender de forma fiável e os resultados caem drasticamente, entrando rapidamente num cenário de *overfitting* (decora os dados em vez de os compreender).
- **Transparência ("Caixa Aberta" vs. "Caixa Fechada"):**
    - **CBR (Transparente):** É como um livro aberto. Se o sistema diagnosticar uma avaria, o operador de manutenção consegue ver exatamente qual foi o motor do passado em que ele se baseou para dar aquela resposta. Isto é vital numa fábrica para criar confiança entre a máquina e o ser humano.
    - **Redes Neuronais (Opacas):** Funcionam como uma "Caixa Negra". São brilhantes a detetar padrões complexos e invisíveis, mas não nos conseguem explicar o seu raciocínio. A justificação perde-se no meio de milhares de cálculos matemáticos.
- **Esforço de Trabalho:** * O **CBR** poupa-nos tempo ao início (não precisa de horas a ser treinado, basta guardar os dados), mas demora um pouco mais no momento de diagnosticar um motor novo, pois tem de o comparar com todo o histórico que guardou.
    - As **Redes Neuronais** dão muito mais trabalho inicial (precisam de ser treinadas exaustivamente e afinadas), mas depois de estarem prontas, o diagnóstico na fábrica é matematicamente instantâneo.

Em suma: O CBR é a escolha ideal quando a fábrica exige que o mecânico compreenda o *porquê* da decisão e precisa de ir adicionando novos casos à memória da máquina facilmente. Já as Redes Neuronais são perfeitas para lidar com bases de dados gigantes e detetar padrões complexos em tempo real, onde a rapidez de resposta importa mais do que a justificação.

6 - Conclusões

A execução do presente projeto permitiu uma análise profunda sobre a operacionalização de modelos de Inteligência Artificial no diagnóstico de sistemas eletromecânicos. A implementação comparativa entre o Raciocínio Baseado em Casos (CBR) e as Redes Neuronais Artificiais (RN) consolidou aprendizagens fundamentais sobre o processamento de informação e a arquitetura de sistemas de apoio à decisão:

- **Determinismo do Pré-processamento:** Confirmou-se a premissa de que a robustez de qualquer modelo preditivo é indissociável da qualidade e integridade dos dados de entrada. O tratamento de valores omissos e a normalização técnica não constituem meras etapas preparatórias, mas sim o alicerce da convergência matemática. A negligência nestes procedimentos

compromete irreversivelmente a consistência dos resultados, independentemente da complexidade do algoritmo utilizado.

- **A Importância das Heurísticas de Domínio:** O desenvolvimento do sistema CBR evidenciou que a eficácia da recuperação de informação depende da tradução correta de leis físicas em métricas matemáticas. A seleção e ponderação de atributos (*Feature Weighting*) baseada no conhecimento especializado permitiu mitigar o ruído estatístico, provando que a integração de variáveis críticas, como a temperatura e a vibração, é essencial para a estabilidade das fronteiras de decisão do modelo.

- **Capacidade de Generalização e Equilíbrio Arquitetural:** Através da experimentação com Redes Neuronais, verificou-se a importância da prevenção do fenómeno de *overfitting* (sobreajuste). Consolidou-se o princípio da moderação: a seleção da arquitetura mínima necessária garante uma maior capacidade de generalização perante dados não observados, evitando que o modelo memorize características do conjunto de treino em detrimento da aprendizagem de padrões universais.

- **Análise de Trade-offs entre Paradigmas:** A comparação entre sistemas permitiu distinguir as vantagens entre modelos de "Caixa-Branca" e "Caixa-Negra". Enquanto o CBR oferece uma total interpretabilidade e auditabilidade das decisões fator crítico em contextos industriais onde a segurança e a justificação técnica são imperativas, as Redes Neuronais garantem uma performance computacional superior em cenários de elevada dimensionalidade e latência reduzida.

Resumindo, o trabalho alcançou os objetivos pedagógicos propostos, demonstrando que a eficácia destas ferramentas de engenharia inteligente reside no equilíbrio entre o rigor estatístico, a otimização algorítmica e a correta integração do conhecimento de domínio no desenho da solução.